NEH, Biased Randomisation and ILS-ESP for the Permutation Flow-Shop Problem: an Adaptive β on Five Taillard Instances

Heuristic Algorithms in Transport and Finance — Master's course assignment
Javier Salvatierra
Jesús Inurria

October 2026

Abstract

The Permutation Flow-Shop Problem (PFSP) asks for the order in which n jobs cross m machines so that the last job finishes as early as possible. We implement the NEH heuristic, its biased-randomised version RandNEH and the iterated local search ILS-ESP of Juan et al. (2014) in Python with Numba, and add a geometric distribution whose parameter β grows after every three failed constructions. On five Taillard instances with 20 to 500 jobs, every RandNEH variant beats NEH on average, by 0.10–0.73 gap points with a biased distribution. The adaptive β matches a well-chosen fixed β except on the 20-job instance, but needs more constructions than $\beta = 0.5$–0.7 (77 against 53 ms on 500 jobs). With a budget of $2nm$ ms, the six ILS-ESP starts end within 0.3 points, close to the noise between seeds.

1 Introduction

A workshop has n jobs and m machines in a line. Every job visits the machines in the same order, and each machine processes the jobs in the same sequence, so the only decision is that sequence: a permutation of the jobs. The goal is to minimise the *makespan*, the time at which the last job leaves the last machine. The search space has $n!$ permutations and the problem is NP-hard for three or more machines [2], hence the use of heuristics. The classical constructive method is NEH, by Nawaz, Enscore and Ham [3].

Juan et al. [1] turn NEH into a multi-start method by *biased randomisation*: instead of always taking the next job of a fixed list, they draw it from a distribution that favours the front of the list (RandNEH), and they use the result as the start of an iterated local search (ILS-ESP). We reimplement both and study two questions: how the choice of distribution affects RandNEH, and whether a better start changes the final ILS-ESP result. We also propose *Geo-adapt*, a geometric distribution whose parameter adapts during the run. We deal only with the makespan, without due dates or setup times.

The test instances are five of Taillard's benchmark [5]: `tai001` ($n \times m = 20 \times 5$), `tai051` (50×20), `tai081` (100×20), `tai101` (200×20) and `tai117` (500×20). The best-known solutions (BKS) come from Taillard's table for `tai001` and from a CSV compilation hosted on Zenodo for the rest, which marks only those of `tai101` and `tai117` as proven optimal. Below we state the model, describe the methods and report two experiments: the constructive phase alone and the complete ILS-ESP.

2 The permutation flow-shop model

Let $p_{j,k}$ be the processing time of job j on machine k ($j = 1,\dots,n$; $k = 1,\dots,m$) and $\pi = (\pi(1),\dots,\pi(n))$ a permutation of the jobs, with Π the set of all of them. With $C_{i,k}$ the time at which the i-th job of the sequence leaves machine k, the PFSP reads

$$\min_{\pi\in\Pi} C_{\max}(\pi) = C_{n,m} \qquad \text{s.t.} \qquad C_{i,k} = \max\big(C_{i-1,k}, C_{i,k-1}\big) + p_{\pi(i),k}, \qquad C_{0,k} = C_{i,0} = 0. \tag{1}$$

Read literally: the i-th job can start on machine k only when the previous job has left that machine and the job itself has left machine $k-1$. There are no other constraints: once the permutation is fixed, the recurrence gives the earliest schedule, so every permutation is feasible and the whole difficulty lies in the order. Evaluating one permutation costs $O(nm)$.

NEH and the local search below repeatedly insert one job at every possible position of a partial sequence of k jobs. Done naively, the $k+1$ evaluations cost $O(k^2 m)$; Taillard's acceleration [4] reuses the completion times of the prefix (forward matrix), the tails of the suffix (backward matrix) and the completion times of the inserted job, and evaluates all positions in $O(km)$. For `tai117` this divides the cost of every insertion step by up to $n = 500$.

3 NEH, RandNEH and ILS-ESP

NEH [3] sorts the jobs by decreasing *total processing time* (TPT), $\sum_k p_{j,k}$, and builds the sequence one job at a time: each job, taken in that order, is inserted at the position of the partial sequence with the lowest partial makespan. It is deterministic: one run, one solution.

RandNEH [1] keeps the insertion step but picks the next job at random from the remaining TPT-sorted list. If r jobs remain and $u \sim U(0,1)$, the position `pos` in the list (0 = first) is drawn from one of three distributions:

- **Triangular** (Tri): $\texttt{pos} = \lfloor r(1-\sqrt{1-u})\rfloor$, with probabilities decreasing linearly along the list. It has no parameter, but its mean position is about $r/3$, so its bias weakens as r grows: 6.17 for $r = 20$, 32.87 for $r = 100$.
- **Geometric** Geo(β), truncated to the r positions, $P(\texttt{pos} = i) \propto \beta(1-\beta)^i$. A higher β concentrates the choice at the front, and once $r \gg 1/\beta$ the mean position barely depends on r: 8.97 for Geo(0.1) and 2.32 for Geo(0.3) with $r = 100$, but 7.81 and 18.45 for Geo(0.05) with $r = 20$ and $r = 100$.
- **Uniform** (Unif), $\texttt{pos} = \lfloor ru\rfloor$, a control with no bias at all.

Table 1: Experiment A, constructive phase: mean gap (%) over 30 runs and mean constructions per run (*Con.*) of the RandNEH loop; NEH is a single deterministic run. BKS in the header. *Mean*: mean gap over the five instances. *ms*: wall-clock time per run on `tai117`.

	`tai001` (1 278)		`tai051` (3 846)		`tai081` (6 171)		`tai101` (11 158)		`tai117` (26 389)			`tai117`
Distribution	Gap	Con.	Gap	Con.	Gap	Con.	Gap	Con.	Gap	Con.	Mean	ms
NEH	0.63	–	6.14	–	6.00	–	3.91	–	1.55	–	3.64	15
Tri	0.37	14.6	5.61	3.0	5.75	14.3	3.77	14.5	1.45	7.9	3.39	143
Geo(0.05)	0.43	16.1	5.51	2.8	5.74	12.1	3.76	13.9	1.44	3.5	3.37	67
Geo(0.1)	0.39	13.8	5.66	2.5	5.61	8.3	3.71	10.6	1.40	2.7	3.36	60
Geo(0.2)	0.28	11.7	5.47	1.7	5.72	5.1	3.72	11.1	1.40	2.2	3.32	51
Geo(0.3)	0.23	8.9	5.55	1.7	5.65	4.9	3.76	7.4	1.40	3.2	3.32	69
Geo(0.5)	0.21	5.3	5.44	1.3	5.68	5.1	3.75	6.9	1.39	2.6	3.29	53
Geo(0.7)	0.10	6.6	5.40	1.2	5.74	6.7	3.71	5.6	1.41	2.4	3.27	53
Unif	0.51	17.0	5.75	4.4	5.87	57.2	3.80	103.6	1.49	149.9	3.48	2 429
Geo-adapt	0.36	14.6	5.58	3.1	5.68	7.2	3.73	11.3	1.41	3.5	3.35	77

A single biased construction is usually worse than NEH: in a preliminary run with 200 single constructions per case, the mean gap was above NEH's in 32 of 35 instance–distribution pairs. RandNEH is therefore a loop (Figure 6 of [1]): compute NEH, then build biased solutions until one is strictly better than NEH or n attempts have been made, and return the best. The output is never worse than NEH, and its cost is the number of constructions.

Geo-adapt is our proposal. A fixed β has to be chosen in advance and the best value depends on the instance. Geo-adapt runs the same loop but changes β with the failures: attempt t uses $\beta_t = \min\bigl(0.7,\ 0.05 + 0.05\lfloor (t-1)/3 \rfloor\bigr)$. It starts wide and moves towards NEH every three failures; a success ends the run, and the next run starts again at 0.05. The only signal used is whether a construction beat NEH.

ILS-ESP [1] starts from a RandNEH solution improved by local search, the *base solution*, and repeats until the time is up:

1. **Enhanced swap.** Swap the jobs at two random positions, then move each of the two jobs to its best position to its left, first the one nearer the front.
2. **Local search** (our choice, see Section 4). Remove every job, in random order, and reinsert it at its best position; repeat until a full pass brings no improvement. The result is a local optimum for insertion moves.
3. **Demon acceptance.** If the new solution is better, it becomes the base solution and its improvement is stored. A worse solution is accepted only right after an improvement and only if it is worse by no more than that improvement, which lets the search leave a local optimum without drifting.

The method has no parameters besides the time budget.

4 Implementation

The code is a single Python notebook, `PFSP_RandNEH_ILS_ESP_en.ipynb` (Python 3.13), with NumPy, pandas, SciPy and matplotlib; every inner loop, from the makespan to the ILS iterations, is compiled with Numba (`@njit`). The key function, `insert_best_position(p, perm, k)`, moves job `perm[k]` to its best position within `perm[0..k]` in $O(km)$; NEH, the local search and the enhanced swap are all built on it. Ties, frequent because makespans are integers, go to the leftmost position, and the TPT sort is stable (lower index first). In the course description we followed, the triangular rule is $X = b(1-\sqrt{1-u})$ with b left undefined; we take $b = r$, so that probabilities depend on the number of eligible jobs. The local search is not detailed either; ours is the insertion local search of Section 3.

Two checks precede the experiments: the accelerated NEH returns the same permutation and makespan as a naive NEH on `tai001` and `tai051` (1 286 and 4 082), and every RandNEH output is a valid permutation whose makespan, recomputed from scratch, matches and never exceeds NEH.

Experiment A (constructive phase) runs NEH once and the RandNEH loop 30 times per distribution and instance with eight fixed distributions — Tri, Geo(β) for $\beta \in \{0.05, 0.1, 0.2, 0.3, 0.5, 0.7\}$ and Unif — plus Geo-adapt. **Experiment B** keeps ILS-ESP intact and changes only the start (NEH, Tri, Geo(0.1), Geo(0.3), Unif, Geo-adapt), with 10 seeds per start and instance and the same budget for all, $2nm$ ms (0.2 s for `tai001`, 20 s for `tai117`), initial construction included. Since Numba has no clock in compiled code, the budget is checked from Python between blocks of about 20 ms of iterations. The 300 runs of Experiment B took 17.2 min on two worker processes of a shared machine. The metric is gap $= 100\,(C_{\max} - \mathrm{BKS})/\mathrm{BKS}$, and each start is compared with NEH by a two-sided Mann–Whitney test, without correction for multiple comparisons. The Geo-adapt parameters were fixed before the runs, after a preliminary check of fixed β from 0.3 to 0.9, and were not tuned.

5 Computational results

Tables 1 and 2 report Experiments A and B.

RandNEH vs. NEH. Every distribution improves on NEH on average in all five instances. With the triangular and geometric distributions, 100 % of the runs beat NEH whenever $n \geq 50$; the uniform control drops to 67 %, 80 % and 97 % on `tai081`, `tai101` and `tai117`. The gain is modest, 0.10–0.73 gap points with a biased distribution (0.06 for the uniform on `tai117`), because the loop stops at the *first* construction that beats NEH, not at the best one. `tai001` is the exception: with $n = 20$ the loop allows only 20 attempts, and the success rate ranges from 33 % (Unif) to 97 % (Geo(0.5) and Geo(0.7)).

Similar quality, different cost. Geo(β) with β between 0.3 and 0.7 forms a plateau: on each instance with $n \geq 50$ the three mean gaps lie within 0.15 points, with 1.2–7.4 constructions per run. The triangular distribution is slightly worse on average (3.39) and needs more constructions (14.3–14.5 on `tai081` and `tai101`, 143 ms against 53 ms for Geo(0.7) on `tai117`), and the uniform is the most expensive (149.9 constructions, 2.4 s): the weaker the bias, the further a construction

Table 2: Experiment B, complete ILS-ESP: final gap (%) as mean ± standard deviation over 10 seeds, and p-value of a two-sided Mann–Whitney test against the NEH start (uncorrected). Budget $2nm$ ms in the header. `tai001` is omitted: all 60 runs reach gap 0. *Mean*: mean final gap over the five instances (`tai001` counted as 0).

	`tai051` (2 s)		`tai081` (4 s)		`tai101` (8 s)		`tai117` (20 s)		
Start	Gap	p	Gap	p	Gap	p	Gap	p	Mean
NEH	1.30 ± 0.09	–	2.40 ± 0.32	–	1.76 ± 0.15	–	0.54 ± 0.13	–	1.20
Tri	1.22 ± 0.22	0.35	2.15 ± 0.25	0.049	1.72 ± 0.14	0.45	0.57 ± 0.11	0.28	1.13
Geo(0.1)	1.33 ± 0.10	0.59	2.17 ± 0.31	0.11	1.78 ± 0.14	0.94	0.59 ± 0.13	0.18	1.17
Geo(0.3)	1.33 ± 0.14	0.75	2.19 ± 0.22	0.089	1.75 ± 0.13	0.91	0.64 ± 0.10	0.063	1.19
Unif	1.12 ± 0.24	0.13	2.32 ± 0.23	0.31	1.75 ± 0.15	0.68	0.62 ± 0.11	0.067	1.16
Geo-adapt	1.29 ± 0.15	0.78	2.11 ± 0.14	0.017	1.75 ± 0.13	0.65	0.62 ± 0.11	0.10	1.16
Iterations	1 439–1 489		604–642		322–338		231–262		

strays from NEH and the less likely it is to beat it. Differences of 0.1–0.2 points between distributions are within the noise of 30 runs and are not a ranking; times come from a shared machine and are only an order of magnitude.

Adaptive β. Geo-adapt gives the same quality as the plateau (mean gap 3.35 against 3.27–3.32) without choosing β, but its cost is that of Geo(0.1)–Geo(0.2), not of a high β: 11.3 constructions on `tai101` against 5.6–6.9 for Geo(0.5)–Geo(0.7), and 77 ms against 53 ms on `tai117`. The β at which each run succeeded shows why: the median is 0.05 on `tai051` and `tai117`, where the first level is usually enough, but 0.175 on `tai101` (range 0.05–0.50), where the first attempts at low β are wasted. Of the 139 successful runs, 134 succeeded at $\beta \leq 0.35$; the maximum was 0.60 and the cap of 0.7 was never reached. The ramp is too slow for small instances: on `tai001` it reaches only $\beta = 0.35$ within the 20 attempts, and 37 % of the runs never beat NEH (63 % success against 87–97 % for Geo(0.3)–Geo(0.7)).

ILS-ESP: the start barely matters. In every instance the six starts end within 0.3 points of each other (0.07 on `tai101`, 0.30 on `tai081`), comparable to the standard deviation between seeds (0.09–0.32). On `tai051` the first local search erases the head start of RandNEH: the mean gap after it is 3.94 from NEH, 4.21 from Tri and 3.92 from Geo-adapt, although they started at 6.14, 5.73 and 5.81 (Experiment B seeds). The 231–1 489 mean iterations per run that follow (Table 2), about 5 000 on `tai001`, then shape the result.

Where the start shows. `tai081` is the only instance where randomised starts help repeatedly: all five end below NEH (2.40) by 0.09–0.30 points, Geo-adapt with the lowest mean (2.11, $p = 0.017$) and the lowest spread between seeds (0.14). Here the local search does not level the starts but widens NEH's deficit, from 0.20–0.33 points before it to 0.36–0.57 after it (4.81 against 4.24–4.45), and the final ranking follows. On `tai117` it is the opposite: NEH leaves the local search among the best (0.94 against 0.91–1.07) and is the best start at the end (0.54 against 0.57–0.64, $p = 0.063$–0.28), with only about 250 iterations in 20 s. Neither result is proof: with 20 non-trivial uncorrected tests, none of the p-values survives a Bonferroni correction, and changing the set of 10 seeds moved the mean final gap of one configuration on `tai117` from 0.71 to 0.63. If we take the best of the 10 seeds, which emulates 10 independent agents run in parallel, the Tri start reaches 0.00, 0.65, 1.72, 1.42 and 0.45 on the five instances. With a short budget and one instance per size we do not compare these gaps with published averages.

6 Conclusions

Biased randomisation turns the deterministic NEH into a loop of cheap constructions that always ends at or below NEH: every distribution beat NEH on average in the five instances, by up to 0.73 gap points. With $\beta \geq 0.1$ the geometric distribution keeps its bias as the number of remaining jobs grows, so on average it matches or improves on the triangular and uniform ones with fewer constructions; a high β keeps each construction close to NEH, which makes a fixed β between 0.3 and 0.7 the cheapest choice (1.2–7.4 constructions per run for $n \geq 50$). Our adaptive β reaches the same quality without choosing β, but costs more than a high fixed β and starts too slowly for $n = 20$. Inside ILS-ESP the constructive gap barely matters: the six starts end within 0.3 points, and the clearest effect of the start is on `tai081`, where NEH's deficit widens in the first local search and all five randomised starts finish below it. A natural next step is a loop that does not stop at the first success, to measure whether the extra constructions pay for themselves.

Code availability. All results come from the notebook `src/PFSP_RandNEH_ILS_ESP_en.ipynb`. By default it reads the saved results in `results/`, so the tables can be reproduced without rerunning the experiments.

References

[1] A. A. Juan, H. R. Lourenço, M. Mateo, R. Luo and Q. Castellà. Using iterated local search for solving the flow-shop problem: parallelization, parametrization, and randomization issues. *Int. Trans. Oper. Res.*, 21(1):103–126, 2014.

[2] M. R. Garey, D. S. Johnson and R. Sethi. The complexity of flowshop and jobshop scheduling. *Math. Oper. Res.*, 1(2):117–129, 1976.

[3] M. Nawaz, E. E. Enscore and I. Ham. A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem. *Omega*, 11(1):91–95, 1983.

[4] E. Taillard. Some efficient heuristic methods for the flow shop sequencing problem. *Eur. J. Oper. Res.*, 47(1):65–74, 1990.

[5] E. Taillard. Benchmarks for basic scheduling problems. *Eur. J. Oper. Res.*, 64(2):278–285, 1993.